Leader-Follower Replication

sync vs async, lag & failover

Watch the 3-min video

In one line: one leader takes every write and streams its log to followers. Whether a commit waits for a follower (sync), for nobody (async) or for any one of several (semi-sync) decides how much data a failover can lose.

1 The problem: confirmed orders that vanish

Your Postgres primary has two replicas. At 2:03 a.m. the primary's disk dies. The last **200 milliseconds** of orders were committed and confirmed to customers, but never reached a replica. You promote a replica, and those orders are simply gone. To see how that happens, look at how replication works and when a commit returns.

2 How leader-follower replication works

One node, the **leader** (also called primary or master), accepts all writes. Every change is first recorded in its **write-ahead log (WAL)**. **Followers** (replicas, standbys) stream that log and replay it in the same order, so they become copies of the leader.

- **Writes** go only to the leader. There is one order of events, so there are no write conflicts.
- **Reads** can go to any node, which is how replicas scale reads (next lesson).
- Postgres, MySQL, SQL Server, MongoDB replica sets and Redis all use this model.

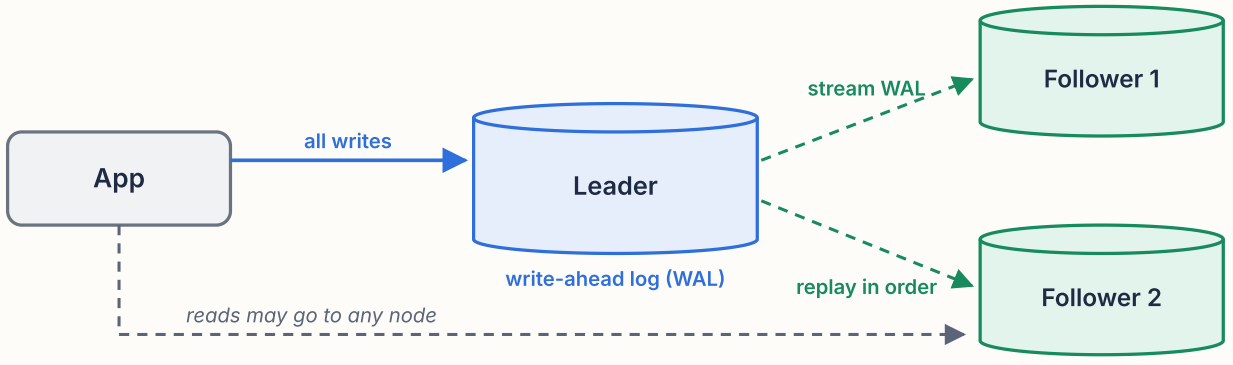


Figure 1. The leader is the single source of truth for writes; followers replay its log in the same order.

3 The big choice: when does a commit return?

- **Synchronous.** The leader waits for a follower to confirm it has the change. No data loss on failover, but every write is slower (at least one network round trip), and if that follower is down, **writes stall**.
- **Asynchronous.** The leader confirms immediately and ships the change afterwards. Fast, but recent writes can be lost if the leader dies.
- **Semi-synchronous.** Wait for **at least one of several** followers. Any one confirmation is enough, so one slow or dead follower does not block writes. This is the common middle ground.



Figure 2. The only difference is the moment the client is told "committed". Everything after that moment is at risk if the leader dies.

Replication lag. With asynchronous replication, followers are always a little behind. Usually it is milliseconds. Under heavy write load, long queries on a replica, or a slow network, it can grow to seconds or even minutes. Monitor it: in Postgres, `pg_stat_replication` on the primary shows each replica's `replay_lag`. Alert on it, because lag is exactly the data you lose in an async failover.

4 Failover: replacing a dead leader

When the leader fails, a follower must take over. Three steps: **detect** the failure (usually a heartbeat timeout), **choose** the most up-to-date follower and promote it, and **redirect** clients to the new leader through a proxy, a virtual IP or DNS. Tools like **Patroni** (Postgres), or managed services like **RDS Multi-AZ**, automate all three.

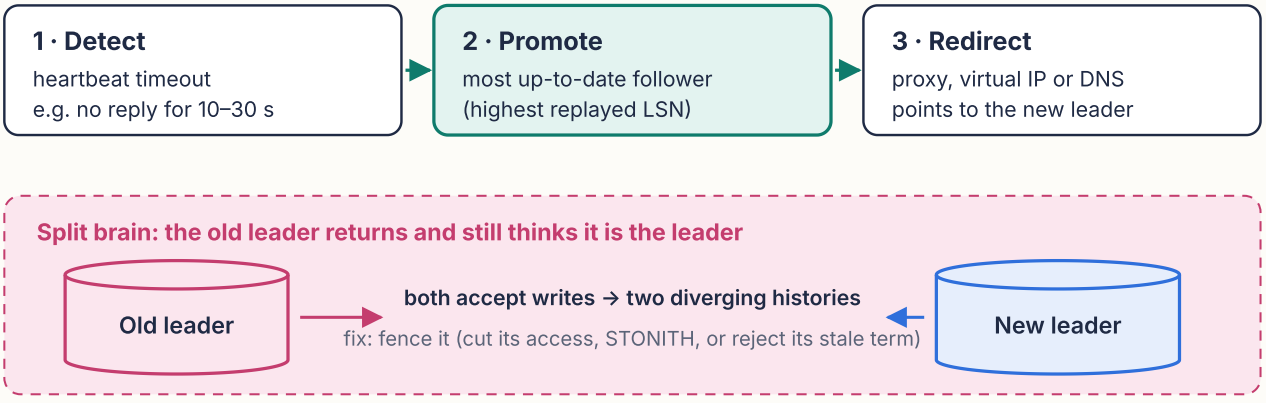


Figure 3. Failover in three steps, and the trap that follows it: an old leader that never got the news.

The traps:

- **Lost writes.** With async replication, writes that never reached the new leader are gone. If the old leader comes back, its extra writes conflict and are usually discarded.
- **Reused IDs.** Auto-increment IDs that the old leader handed out may be handed out again by the new one, colliding with data already in caches or other systems.
- **Split brain.** If the old leader returns still believing it is the leader, two nodes accept writes. It must be **fenced off**.
- **The timeout.** Too short causes needless failovers on a brief network blip; too long means longer downtime.

5 Back to the example: protect the orders

A primary with two async replicas is fast, but can lose recent writes on failover. For orders, make the commit wait for **any one** of the two replicas:

```
# postgresql.conf on the primary
synchronous_commit = on
synchronous_standby_names = 'ANY 1 (replica1, replica2)'
```

Each commit now waits for one confirmation. A promoted replica (the one that confirmed) always has every confirmed order, and one replica can still fail without blocking writes.

WORKED EXAMPLE: WHAT ASYNC COSTS, WHAT SYNC COSTS

- **Async loss:** at 500 orders/s and 200 ms of lag, a failover loses about 500 × 0.2 = **100 confirmed orders**.
- **Sync cost:** each commit adds one round trip to a replica: about **1 ms** in the same data centre, but **tens of ms** across regions.
- **Why ANY 1 of 2:** with a single named sync replica, losing that replica stalls all writes. With two candidates, one can die and commits still return.

COMMON MISTAKES

- Assuming replicas mean no data loss: with async replication they do not.
- Synchronous replication to a single replica, so its outage stops all writes.
- Failing over without fencing the old leader: split brain.
- Not monitoring replication lag, so nobody knows how much a failover would lose.

6 Key terms

Leader / primary: the one node that accepts writes

Follower / replica: replays the leader's log

WAL: write-ahead log: ordered record of changes

Synchronous: commit waits for a follower

Asynchronous: commit returns at once

Semi-sync: wait for any one of several

Replication lag: how far a follower is behind

Failover: promoting a follower to leader

Split brain / fencing: two leaders / cutting off the old one

7 Interview questions

Q1. What happens to unreplicated writes during failover? BASIC

With async replication, writes the old leader acknowledged but had not yet shipped to the promoted follower are lost. If the old leader returns, its conflicting writes are usually discarded. Prevent it with sync or semi-sync replication for critical data, promote the most up-to-date replica, and fence the old leader.

Q2. Sync vs async replication: what is the trade-off? BASIC

Sync: no loss, but every write waits for a replica round trip and stalls if that replica is down. Async: fast commits, but anything inside the lag window (often ms, sometimes seconds) can be lost.

Q3. How do you monitor replication lag in Postgres? BASIC

Query pg_stat_replication on the primary and look at replay_lag (or compare WAL positions). On a replica, now() − pg_last_xact_replay_timestamp() gives lag as a time. Alert when it exceeds, say, a few seconds.

Q4. A payments database runs async with 2 replicas. The business says no confirmed payment may ever be lost. What do you change? SCENARIO

Use semi-sync: synchronous_standby_names = 'ANY 1 (r1, r2)'. Each commit waits for one replica (about 1 ms in-region), one replica can fail without stalling writes, and on failover promote the replica that is most up to date.

Q5. Your failover timeout is 3 s and you see several failovers a week with no real outage. Why, and what do you do? SCENARIO

Brief network blips or GC pauses exceed 3 s, so healthy leaders are being replaced, with lost writes each time under async. Raise the timeout (e.g. to 10–30 s), require several missed heartbeats, and use a consensus-based tool like Patroni.

Q6. After a failover, users report orders with IDs that point to the wrong customer in the cache. What happened? SCENARIO

The old leader handed out IDs that never replicated. The new leader reused them, so cached entries keyed by the old IDs now collide. Invalidate caches on failover, and prefer IDs that cannot be reused (UUIDs, or sequences advanced past the old maximum).

Q7. Does synchronous replication to one replica guarantee zero downtime? TRICKY

No. It guarantees no data loss, but if that replica is down, writes stall, which is downtime of a different kind. That is why semi-sync waits for any one of several replicas.

Q8. The old leader comes back after failover. What must happen? TRICKY

It must be fenced: blocked from taking writes (STONITH, revoked access, or a stale term number it cannot use). Then it rejoins as a follower, rewinding any writes the new leader never had (e.g. pg_rewind).

Next: SD-E-020 · Read Replicas & Replication Lag backendroadmap.com